

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ
ІВАНА ФРАНКА

Факультет прикладної математики та інформатики

Кафедра дискретного аналізу та інтелектуальних систем

КУРСОВА РОБОТА

«Дослідження стійкості стеганографічних алгоритмів LSB та DWT-SVD до шумових і компресійних атак»

Виконала: студентка групи ПМІ-35
спеціальності 122 – комп'ютерні науки

Пелешак Вероніка Русланівна

(підпис)

(прізвище та ініціали)

Керівник Прядко Ольга Ярославівна

(підпис)

(прізвище та ініціали)

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ	3
ВСТУП.....	4
РОЗДІЛ 1 ТЕОРЕТИКО-МЕТОДОЛОГІЧНІ ОСНОВИ	
СТЕГАНОГРАФІЧНОГО ЗАХИСТУ АУДИОДАНИХ	5
1.1 Аналіз сучасного стану та методів цифрової стеганографії.....	5
1.2 Огляд та аналіз аналогічних програмних розробок	7
1.3 Математичні моделі алгоритмів DWT-SVD та LSB	8
1.4 Застосування завадостійкого кодування Хеммінга та Ріда-Соломона	12
РОЗДІЛ 2 ПРАКТИЧНА РЕАЛІЗАЦІЯ ПРОГРАМНОГО КОМПЛЕКСУ	15
2.1 Архітектура програмного комплексу та взаємодія компонентів.....	15
2.2 Розробка модуля симуляції кібератак та імплементація метрик оцінювання.....	17
РОЗДІЛ 3 АНАЛІЗ РЕЗУЛЬТАТІВ ТЕСТУВАННЯ АЛГОРИТМІВ	20
3.1 Оцінка ефективності методів завадостійкого кодування	20
3.2 Дослідження стійкості алгоритмів на різних типах аудіосигналів	22
3.3 Підсумковий аналіз результатів тестування.....	26
ВИСНОВКИ.....	28
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	29

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

AWGN (Additive White Gaussian Noise) — адитивний білий гаусівський шум.

BER (Bit Error Rate) — коефіцієнт бітових помилок; показник, що визначає відсоток втрачених або спотворених бітів після витягнення даних.

DWT (Discrete Wavelet Transform) — дискретне вейвлет-перетворення, математичний апарат частотної області.

HAS (Human Auditory System) — слухова система людини.

LSB (Least Significant Bit) — метод найменш значущого біта; алгоритм просторової області для стеганографічного вбудовування.

NCC (Normalized Cross-Correlation) — нормалізована кореляція; метрика, що показує ступінь подібності між оригінальним і витягнутим водяним знаком.

PSNR (Peak Signal-to-Noise Ratio) — пікове відношення сигналу до шуму (метрика оцінки акустичної прозорості).

SNR (Signal-to-Noise Ratio) — відношення сигналу до шуму.

SVD (Singular Value Decomposition) — сингулярний розклад матриці.

ВСТУП

У сучасному інформаційному суспільстві обсяг цифрового мультимедійного контенту, особливо аудіоданих, стрімко зростає. Цифрові аудіофайли легко копіювати, змінювати та поширювати онлайн, що створює проблему в надійному захисті авторських прав та безпечних способах обміну інформацією.

Одним із традиційних методів захисту конфіденційних даних є криптографія. Ця техніка перетворює повідомлення на зашифрований текст. Але її головним недоліком є те, що вона не приховує передачу будь-якої інформації. Іншим методом є цифрова стеганографія, яка є наукою про приховування факту передачі даних шляхом вбудовування секретного повідомлення або цифрового водяного знака у звичайний, загальнодоступний файл [13].

На ринку існують популярні стеганографічні інструменти, такі як SilentEye і DeepSound [2, 3]. Проте більшість із них використовують базовий метод заміни молодшого біта без алгоритмів корекції помилок. Як наслідок, після конвертації аудіо у формат MP3, прихована інформація може бути втрачена.

Мета роботи — розробити власний програмний комплекс, який зможе надійно приховувати текст у аудіоконтейнери. Робота фокусується на двох різних підходах: простому (LSB) та складному гібридному (DWT-SVD). Крім того, передбачено розробку модуля тестування, призначеного для моделювання атак та оцінки стійкості алгоритмів.

РОЗДІЛ 1 ТЕОРЕТИКО-МЕТОДОЛОГІЧНІ ОСНОВИ СТЕГANOГРАФІЧНОГО ЗАХИСТУ АУДІОДАНИХ

1.1 Аналіз сучасного стану та методів цифрової стеганографії

Для розуміння того, як працюють стеганографічні алгоритми, необхідно розглянути загальний принцип побудови такої системи. Вона базується на взаємодії трьох основних компонентів: оригінального аудіофайлу, секретного повідомлення та спеціального ключа, який забезпечує безпеку вбудовування [13].

Весь процес можна розділити на два ключові етапи. Перший етап — вбудовування: програма за допомогою алгоритму та ключа вбудовує секретний текст в оригінальний аудіофайл. У результаті отримуємо файл, який не відрізняється від оригіналу, що дозволяє безпечно передавати його відкритими каналами зв'язку. Другий етап — витягнення: відбувається на стороні отримувача, який, маючи той самий секретний ключ, проводить зворотну операцію та відновлює прихований текст.

Під час передачі даних файл часто піддається стисненню або впливу перешкод. Через це отримувач може отримати пошкоджений файл, а витягнутий текст може містити помилки. Саме тому варто використовувати завадостійке кодування, яке дозволяє виявляти та виправляти пошкоджені біти.

Аудіоконтейнери вважаються одними з найскладніших для приховування даних. Все тому, що слухова система людини, HAS, є надзвичайно чутливою до будь-яких, навіть найменших, шумів та спотворень [14, 16]. Щоб зробити втручання непомітним, алгоритми використовують властивості психоакустики. Наприклад, інформація може вбудовуватися у фрагменти аудіосигналу з високою гучністю, де зміни є менш помітними для людського слуху.

Сучасні алгоритми аудіостеганографії можна глобально поділити на три основні категорії [16]:

1. **Методи просторової (часової) області:** ці методи працюють безпосередньо зі звуковою хвилею — алгоритм маніпулює цифровими значеннями гучності відліків. Це найпростіший і найшвидший підхід.
2. **Трансформаційні (частотні) методи:** звук розкладається на спектри та частоти, і дані вбудовуються у частотні компоненти сигналу. Ці методи складніші, але набагато надійніші.
3. **Структурні методи та специфічні формати:** інформація ховається не в самому звуці, а у службових даних — наприклад, у полях заголовка файлу або безпосередньо в параметрах стиснення аудіокодека.

Оцінка ефективності будь-якого розробленого алгоритму здійснюється за трьома базовими параметрами [17]:

1. **Прозорість:** міра того, наскільки стегоконтейнер відрізняється від оригіналу. Оцінюється за допомогою фізичних метрик: відношення сигналу до шуму (SNR) та пікового відношення сигналу (PSNR).
2. **Стійкість:** здатність алгоритму витягувати водяний знак після впливу зовнішніх перешкод або навмисних атак. Оцінюється за допомогою коефіцієнта бітових помилок (BER) та нормалізованої взаємної кореляції (NCC).
3. **Ємність:** максимальна кількість бітів, що може бути прихована в одній секунді треку без порушення прозорості.

Покращення одного з цих параметрів неминуче призводить до погіршення інших.

Головна відмінність між просторовими та частотними методами є співвідношення швидкості та надійності. Просторові алгоритми працюють швидко, бо вони записують дані напряму у файл, але через це вони вразливіші до пошкоджень. Натомість трансформаційні методи є стійкішими, тому що ховають інформацію на рівні звукових частот. Це краще захищає дані від знищення, але

потрібно більше часу на розрахунки. На основі цього було вибрано такі алгоритми для порівняння: просторовий LSB та гібридний частотний DWT-SVD. Оскільки вони є достатньо різними, це дозволяє краще оцінити переваги та недоліки обох підходів в різних умовах.

1.2 Огляд та аналіз аналогічних програмних розробок

У процесі дослідження було виявлено, що сучасних повноцінних веб-сервісів для приховування інформації в аудіофайлах існує небагато. Більшість існуючих програм є десктопними додатками, які мають обмеження: вони або платні, або орієнтовані виключно на роботу з фотографіями, або є застарілими. Крім того, більшість утиліт розроблені виключно під операційну систему Windows.

Програма DeepSound є однією з найвідоміших десктопних утиліт для аудіостеганографії, створеною для операційної системи Windows [2]. Вона дозволяє ховати секретні файли всередині звукових доріжок таких форматів як WAV та FLAC. Принцип роботи цього додатка базується на заміні бітів у просторовій області аудіосигналу. Головною перевагою є інтегрований криптографічний захист та підтримка контейнерів без втрат якості. Також варто врахувати, що алгоритм модифікує відліки амплітуди, тому приховані дані є вразливими до стиснення із втратами.

Ще одним відомим інструментом є утиліта SilentEye [3]. Завдяки системі плагінів програма дозволяє приховувати інформацію у цифрових зображеннях та в аудіофайлах формату WAV. Програма має модульну архітектуру та підтримує роботу з різними типами медіафайлів. Проте її аудіомодуль використовує класичний метод LSB, без інтегрованих механізмів виправлення помилок. І так як проект тривалий час не оновлювався, під час запуску на сучасних апаратних архітектурах, зокрема ARM, можуть виникати проблеми із сумісністю бібліотек.

Зведені результати порівняльного аналізу розглянутих програмних аналогів та розробленого комплексу наведено у таблицю 1.1.

Таблиця 1.1 — Порівняльний архітектурний аналіз існуючих стеганографічних засобів та розробленого комплексу

Критерій порівняння	DeepSound	SilentEye	Розроблений комплекс
Архітектура та інтерфейс	Десктоп	Десктоп	Веб-додаток
Підтримувані ОС	Тільки Windows	Windows, macOS(Intel), Linux	Універсальна, через браузер
Алгоритм вбудовування	Просторові методи	Просторовий (LSB)	LSB + DWT-SVD
Завадостійке кодування	Відсутнє	Відсутнє	Реалізовано
Модуль імітації кібератак	Відсутній	Відсутній	Інтегровано
Розрахунок наукових метрик	Немає	Немає	Присутній (BER, NCC, SNR, PSNR)

Аналіз наведених у таблиці 1.1 даних демонструє, що існуючі рішення здебільшого використовують просторові методи та не мають завадостійкого кодування. Крім того, усі розглянуті аналоги є десктопними програмами, що вимагають локального встановлення. Це обґрунтовує доцільність розробки власного веб-додатка, який поєднає зручність використання у браузері із застосуванням стійкого алгоритму DWT-SVD та захисту від помилок.

1.3 Математичні моделі алгоритмів DWT-SVD та LSB

У даній роботі використовуються два методи для приховування інформації в аудіоданих: LSB та DWT-SVD.

Цифровий аудіофайл складається з послідовності числових значень — амплітуд звуку. Основний принцип методу LSB полягає в тому, що при змінні

наймолодшого біта у двійковому поданні амплітуди зміни гучності є настільки незначними, що вони практично не сприймаються на слух [16].

У розробленому програмному модулі ця операція виконується шляхом заміни наймолодших бітів оригінального звуку на біти секретного тексту. Позначимо через s_i оригінальне числове значення амплітуди, а через w_i — біт секретного повідомлення, тобто 0 або 1. Зміна амплітуди виконується за допомогою швидких побітових операторів:

$$s'_i = (s_i \text{ AND } \sim 1) \text{ OR } w_i \quad (1.1)$$

Програма спочатку застосовує логічне множення для обнулення останнього біта в оригінальному числі, а потім використовує логічне додавання для запису біта секретного повідомлення на порожнє місце. Даний підхід дозволяє сховати дані без складних математичних обчислень, зберігаючи майже непомітне спотворення звуку.

Схему процесу заміни бітів методу LSB наведено на рисунку 1.1.

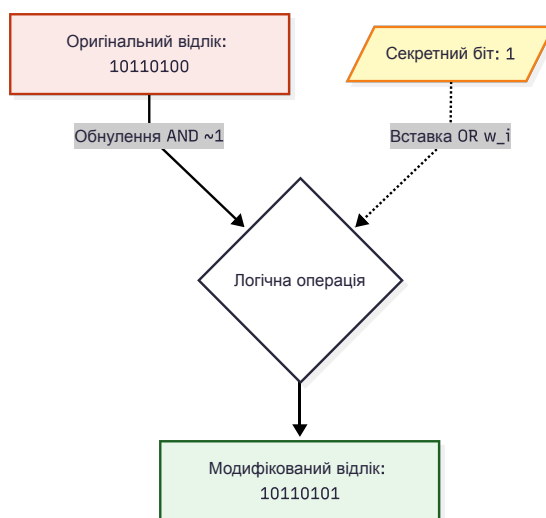


Рисунок 1.1 — Візуалізація процесу вбудовування даних методом LSB

Процес вилучення прихованої інформації у методі LSB не потребує наявності оригінального аудіофайлу. Позначимо через s'_i значення амплітуди отриманого аудіофайлу, а через w'_i — витягнутий біт повідомлення. Операція зчитування виконується шляхом побітового логічного множення на одиницю:

$$w'_i = s'_i \text{ AND } 1 \quad (1.2)$$

Ця операція повертає значення лише останнього біта кожного відліку. Послідовність бітів w'_i конвертується назад у текстовий формат, що дозволяє отримати початкове секретне повідомлення.

На відміну від LSB, метод DWT-SVD працює не з амплітудою звуку, а з його частотним представленням [17]. Цей процес реалізований у два етапи.

На першому етапі програма пропускає звуковий сигнал через систему цифрових фільтрів, розділяючи його на дві складові. Перша — це апроксимуючі коефіцієнти, які містять низькі частоти, тобто основну енергію звуку. Друга — деталізуючі коефіцієнти, що містять високі частоти, шум. Так як алгоритми стиснення зазвичай зрізають саме високі частоти, секретний текст вбудовується в масив низьких частот.

На другому етапі отриманий масив низьких частот алгоритм перетворює на матрицю і виконує її сингулярний розклад [15]. Математичний зміст цього перетворення полягає в знаходженні сингулярних чисел матриці, що характеризують її структуру. Позначимо матрицю цих чисел як Σ . Їхня головна властивість — стабільність: якщо аудіофайл стиснути або додати шум, ці числа майже не зміняться.

Процес приховування даних відбувається шляхом додавання бітів тексту до цих значень. Позначимо через Σ' нові сингулярні числа, через W — масив нашого секретного тексту, а через α — коефіцієнт інтенсивності. Формула вбудовування має такий вигляд:

У даному випадку коефіцієнт α визначає інтенсивність вбудовування: чим більше значення, тим краще захищений текст, але тим помітніше спотворюється оригінальний звук. У програмі він дорівнює 0.005.

Після модифікації сингулярних чисел алгоритм проводить зворотні перетворення, збираючи звук знову у часову хвилю для створення готового аудіофайлу.

Схему процесу вбудовування даних методу DWT-SVD наведено на рисунку 1.2.

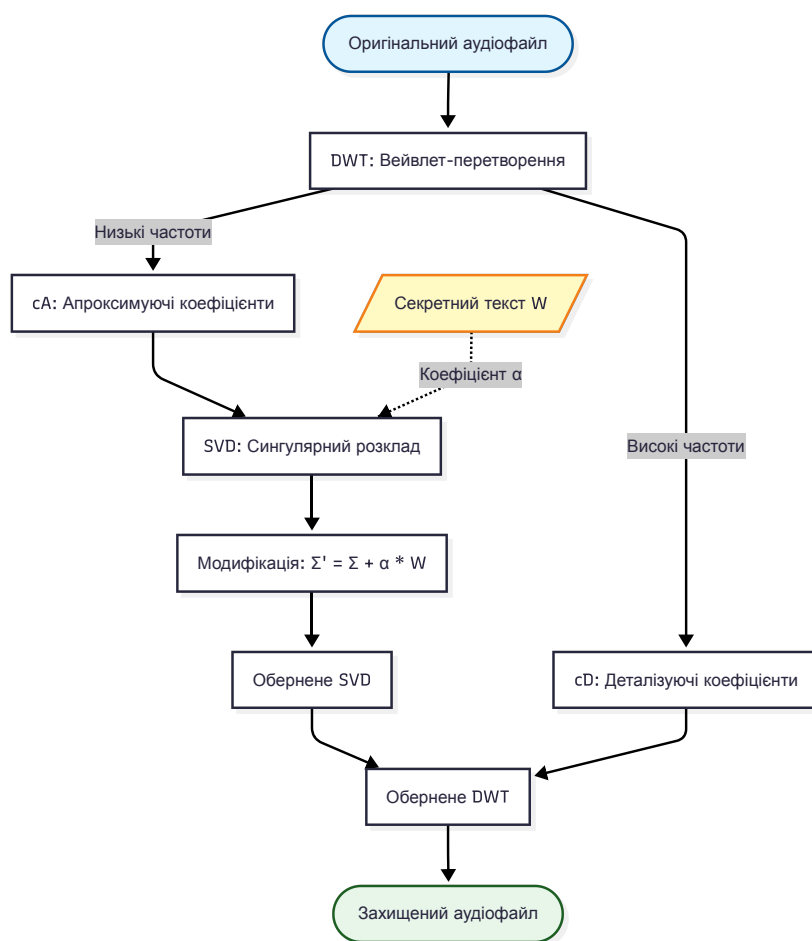


Рисунок 1.2 — Структурна схема алгоритму DWT-SVD

Для вилучення секретної інформації алгоритм знову обчислює сингулярні числа для обох файлів: оригінального та отриманого файлу з секретним текстом.

Відновлення тексту відбувається шляхом знаходження різниці між цими числами за оберненою формулою:

$$W' = \frac{\Sigma^* - \Sigma}{\alpha} \quad (1.4)$$

Завдяки використанню матричних перетворень та вбудовуванню інформації в найбільш стабільні компоненти звукового спектра, цей алгоритм дозволяє програмі відновлювати текст майже без помилок навіть після стиснення файлу чи накладання шумових перешкод.

1.4 Застосування завадостійкого кодування Хеммінга та Ріда-Соломона

Будь-які деструктивні впливи на аудіофайл призводять до виникнення інвертованих бітів під час процесу вилучення прихованих даних. Щоби зменшити кількість таких помилок у розробленому програмному комплексі застосовується алгоритм прямої корекції помилок [1]. Головний принцип полягає в тому, що до секретного повідомлення перед приховуванням додається захисна інформація. Просторові та частотні методи стеганографії спричиняють різні типи помилок, тому у системі реалізовано два алгоритми: лінійні блокові коди Хеммінга для методу просторової області та недвійкові циклічні коди Ріда-Соломона для частотної області.

При кодуванні алгоритмом Хеммінга секретне повідомлення поділяється на групи по чотири інформаційні біти. Позначимо ці біти як d_0, d_1, d_2, d_4 . Для їхнього захисту алгоритм генерує три перевіірочні біти, які позначимо як p_1, p_2, p_3 . Математична модель розрахунку цих бітів описується такою системою рівнянь:

$$p_1 = (d_0 + d_1 + d_3) \pmod{2} \quad (1.5)$$

$$p_2 = (d_0 + d_2 + d_3) \pmod{2} \quad (1.6)$$

$$p_3 = (d_1 + d_2 + d_3) \pmod{2} \quad (1.7)$$

Після обчислення перевірочних бітів формується єдиний семибітний блок. Код Хеммінга здатен виправити лише одну помилку в межах одного блоку. Тому перед приховуванням усі згенеровані біти переставляються за допомогою генератора псевдовипадкових чисел. Завдяки цьому пошкодження в аудіофайлі під час декодування розподіляються між блоками у вигляді окремих помилок, які код Хеммінга може виправити.

Під час вилучення даних отримувач зчитує семибітний блок інформації, який міг зазнати спотворень. Позначимо отримані біти цього блоку як $c_0, c_1, c_2, c_3, c_4, c_5, c_6$. Для виявлення та локалізації помилок обчислюється вектор синдрому, ненульове значення якого вказує на позицію спотвореного елемента в блоці. Позначимо компоненти синдрому як s_1, s_2, s_3 . Обчислення синдрому відбувається за таким алгоритмом:

$$s_1 = (c_0 + c_2 + c_4 + c_6) \pmod{2} \quad (1.8)$$

$$s_2 = (c_1 + c_2 + c_5 + c_6) \pmod{2} \quad (1.9)$$

$$s_3 = (c_3 + c_4 + c_5 + c_6) \pmod{2} \quad (1.10)$$

Після цього алгоритм застосовує оператор логічної інверсії до пошкодженого елемента, відновлюючи оригінальне повідомлення.

Для частотного алгоритму використання коду Хеммінга є неефективним, тому що матричні перетворення генерують пакетні помилки. Тому для DWT-SVD було застосовано код Ріда-Соломона [1]. Його головна відмінність полягає в тому, що він працює не з окремими бітами, а з цілими байтами інформації.

У розробленому програмному модулі це кодування реалізоване за допомогою спеціалізованої бібліотеки reedsolo [9], яка виконує обчислення в розширеному полі Галуа. Позначимо секретне повідомлення як інформаційний поліном $m(x)$, генераторний поліном, який відповідає за створення перевірочних символів, як $g(x)$, а захищене кодове слово як поліном $c(x)$. Процес кодування виконується так, щоб сформований поліном ділився на генераторний без залишку. Ця операція описується рівнянням, де залишок від ділення виконує функцію захисних байтів:

$$c(x) = m(x) \cdot x^{2t} + (m(x) \cdot x^{2t} \bmod g(x)) \quad (1.11)$$

У конфігурації програмного модуля встановлено додавання десяти захисних байтів для кожного інформаційного блоку.

Під час декодування алгоритм спочатку перевіряє, чи містить отримане повідомлення помилки. Оскільки кодування відбувається на рівні байтів, а не окремих бітів, цей метод ефективно захищає від пакетних помилок. Система обчислює точне місце розташування пошкодженого байта шляхом аналізу надлишкових захисних символів, доданих на етапі кодування, та повністю відновлює початковий текст.

РОЗДІЛ 2 ПРАКТИЧНА РЕАЛІЗАЦІЯ ПРОГРАМНОГО КОМПЛЕКСУ

2.1 Архітектура програмного комплексу та взаємодія компонентів

Мовою програмування для реалізації програмного комплексу було обрано Python через велику кількість спеціалізованих бібліотек для обробки даних. А його вихідний код розміщено у відкритому репозиторії GitHub [12]. Зокрема, було використано такі бібліотеки як SciPy та PyWavelets [7, 8]. Бібліотека SciPy застосовується для цифрової обробки сигналів та моделювання частотних фільтрів. А PyWavelets використовується для виконання дискретного вейвлет-перетворення. Ця бібліотека є основою методу DWT-SVD та за допомогою неї сигнали розкладаються на апроксимуючі та деталізуючі коефіцієнти.

Архітектуру програмного комплексу побудовано за клієнт-серверним патерном. Клієнтська частина розроблена на базі фреймворку Streamlit [4], який дозволяє створювати інтерактивний веб-інтерфейс для введення параметрів і відображення результатів аналізу без потреби у складних клієнтських системах. Серверна обробка та взаємодія між модулями реалізовані за допомогою фреймворку Django [10]. Логіка обробки даних у системі є послідовною: після отримання HTTP-запиту із WAV-контейнером та текстом, сервер послідовно виконує завадостійке кодування повідомлення, вбудовує його у часову або частотну область сигналу та повертає готовий результат користувачеві або зберігає розраховані метрики в базі даних.

Взаємодія між клієнтською та серверною частинами реалізується через REST API контролер [5], який надає три основні маршрути:

1. **Ендпоінт вбудовування:** приймає оригінальний файл і текст, маршрутизує запит до відповідного алгоритму, застосовує вибраний тип завадостійкого кодування та розраховує ємність вбудовування. Одразу після створення

стежоконтейнера сервер обчислює метрики: SNR та PSNR — і зберігає їх у базу даних.

2. **Ендпоінт експертизи:** дозволяє проводити тестування. Контролер приймає конфігурацію кібератак, застосовує їх до аудіоданих, вилучає водяний знак та повертає статистику з показниками BER та NCC.
3. **Ендпоінт бенчмаркінгу:** програма тестує захищений трек через масив із шести деструктивних впливів, паралельно обчислюючи всі метрики.

Структурна схема взаємодії компонентів системи наведена на рисунку 2.1.

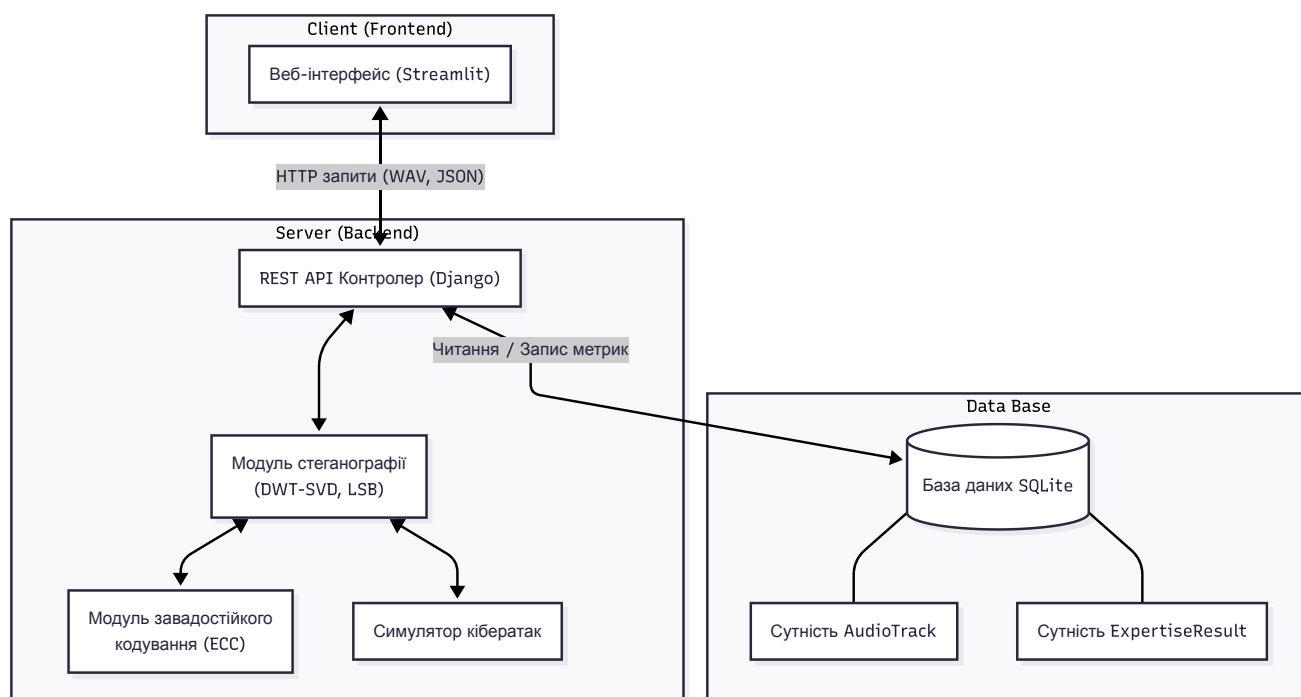


Рисунок. 2.1 — Архітектура програмного комплексу

Для збереження метаданих експертиз спроектовано реляційну базу даних під управлінням СУБД SQLite. Логічна структура бази даних наведена на рисунку 2.2.

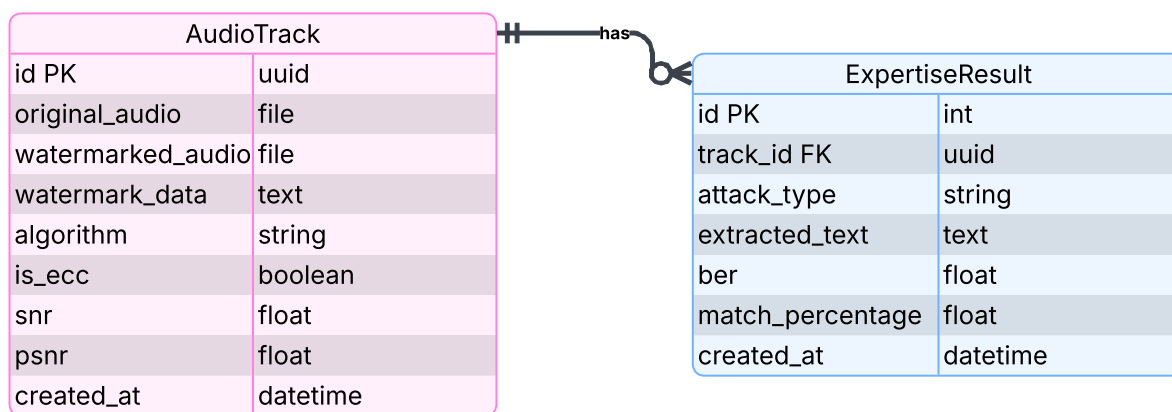


Рисунок 2.2 — ER-діаграма бази даних системи

Логічна структура системи складається з двох сутностей, пов'язаних відношенням «один-до-багатьох». Модель AudioTrack зберігає основні дані про захищений аудіофайл: шляхи до файлів, текст водяного знаку, тип алгоритму та метрики SNR і PSNR. Сутність ExpertiseResult використовується для збереження результатів перевірки стійкості. Оскільки один аудіофайл може перевірятися різними атаками, вона містить окремі записи з інформацією про тип атаки, значення BER, NCC та вилучений текст без дублювання основних даних аудіотреку.

2.2 Розробка модуля симуляції кібератак та імплементація метрик оцінювання

Для проведення комплексного дослідження було спроектовано та реалізовано модуль автоматизованої експертизи. Головна задача цього модуля полягає в імітації деструктивних впливів та обчисленні метрик.

Усі дії з аудіоданими під час атак виконуються в оперативній пам'яті сервера. Це реалізовано за допомогою віртуальних потоків `io.BytesIO`, куди завантажуються масив байтів стегоконтейнера перед обробкою.

У модулі реалізовано симуляцію п'яти типів атак. До базових перетворень належать низькочастотна фільтрація із застосуванням фільтра Баттерворта, який реалізований за допомогою модуля `scipy.signal` [11], амплітудне масштабування та усічення фрагментів аудіопотоку. Також у модулі розроблено такі типи атак як шумові та компресійні.

Для оцінки стійкості алгоритмів у систему додано обчислення чотирьох базових метрик. SNR та PSNR вимірюють акустичну прозорість і рівень деградації самого аудіоконтейнера, а BER та NCC дозволяють математично оцінити ступінь руйнування прихованого повідомлення.

SNR (Signal-to-Noise Ratio) — це відношення енергії оригінального сигналу до енергії внесених спотворень. Якщо $SNR > 20$ дБ, то спотворення є малопомітними, а при $SNR \geq 30$ дБ внесені спотворення стають практично непомітними.

Позначимо через s_i значення амплітуди i -го відліку оригінального сигналу, через s'_i — значення i -го відліку захищеного стегоконтейнера, а через N — загальну кількість дискретних відліків.

Математично SNR обчислюється за формулою [14, 15]:

$$SNR = 10 \log_{10} \left(\frac{\sum_{i=1}^N s_i^2}{\sum_{i=1}^N (s_i - s'_i)^2} \right) \quad (2.1)$$

PSNR (Peak Signal-to-Noise Ratio) — оцінює рівень шуму відносно максимально можливої амплітуди сигналу. Першим кроком для знаходження цієї метрики є розрахунок середньоквадратичної похибки:

$$MSE = \frac{1}{N} \sum_{i=1}^N (s_i - s'_i)^2 \quad (2.2)$$

Після цього PSNR обчислюється за формулою, де MAX_I — максимально можливе абсолютне значення амплітуди відліку [15]:

$$PSNR = 20 \log_{10} \left(\frac{MAX_I}{\sqrt{MSE}} \right) \quad (2.3)$$

Так як в системі використовуються аудіофайли формату WAV, діапазон становить від -32768 до 32767. Отже, $MAX_I = 32767$.

BER (Bit Error Rate) — відношення кількості неправильно витягнутих символів до загальної довжини повідомлення. В ідеальній системі $BER = 0$. Процес виявлення розбіжностей описується через логічну операцію XOR для кожного біта.

Позначимо через L довжину бінарного водяного знаку, через $w_i \in \{0, 1\}$ — j -й біт оригінального повідомлення та через $w'_j \in \{0, 1\}$ — j -й біт витягнутого повідомлення. Тоді BER обчислюється за формулою [17]:

$$BER = \frac{1}{L} \sum_{j=1}^L (w_j \oplus w'_j) \times 100 \% \quad (2.4)$$

NCC (Normalized Cross-Correlation) — коефіцієнт кореляції, що показує структурну схожість між оригінальним і витягнутим водяними знаками у діапазоні від 0 до 1. Для отримання оцінки цієї схожості використовується наступне математичне відношення [17]:

$$NCC = \frac{\sum_{j=1}^L (w_j \cdot w'_j)}{\sqrt{\sum_{j=1}^L w_j^2 \cdot \sum_{j=1}^L (w'_j)^2}} \quad (2.5)$$

РОЗДІЛ 3 АНАЛІЗ РЕЗУЛЬТАТІВ ТЕСТУВАННЯ АЛГОРИТМІВ

3.1 Оцінка ефективності методів завадостійкого кодування

Для перевірки роботи алгоритмів виправлення помилок було створено тестовий аудіосигнал із додаванням слабого шуму. А секретним повідомленням було обрано «secretText». Такий сигнал генерує контрольовану кількість дрібних випадкових помилок. Це дає змогу перевірити саму здатність кодів знаходити та відновлювати пошкоджені дані, перш ніж тестувати їх в більш складних умовах.

Таблиця 3.1 — Результати тестування частотного алгоритму DWT-SVD без завадостійкого кодування

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	25.3	28.33	secretText
Шум (AWGN) 30 dB	0.9	10.0	24.17	27.2	3secretText
Шум (AWGN) 15 dB	0.9	10.0	15.37	18.4	?secretText
Фільтр 4000 Hz	1.0	0.0	30.22	33.25	secretText
Обрізання 10%	0.9	10.0	9.89	12.91	3secretText
MP3 Стиснення 128kbps	0.9	10.0	22.48	25.51	3secretText
MP3 Стиснення 64kbps	0.9	10.0	21.57	24.6	3secretText

Таблиця 3.2 — Результати тестування частотного алгоритму DWT-SVD із застосуванням коду Ріда-Соломона

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	25.3	28.33	secretText
Шум (AWGN) 30 dB	1.0	0.0	24.18	27.21	secretText
Шум (AWGN) 15 dB	1.0	0.0	15.34	18.37	secretText
Фільтр 4000 Hz	1.0	0.0	30.22	33.25	secretText
Обрізання 10%	1.0	0.0	9.89	12.91	secretText
MP3 Стиснення 128kbps	1.0	0.0	22.5	25.53	secretText
MP3 Стиснення 64kbps	1.0	0.0	21.79	24.81	secretText

Аналіз даних у таблиці 3.1 показує, що під впливом таких факторів, як шум, обрізання сигналу та стиснення MP3, сингулярні числа матриці зазнають невеликих спотворень. Хоча структура водяного знака в цілому зберігається, це призводить до бітових помилок із $BER = 10\%$, що проявляється у пошкоджені тексті.

Натомість результати таблиці 3.2 показують важливість використання коду Ріда-Соломона. Завдяки додатковим захисним символам система знаходить і виправляє пошкоджені ділянки. У результаті після тих самих атак BER зменшується до нуля, NCC дорівнює 1, а вихідне повідомлення відновлюється без втрат.

Таблиця 3.3 — Результати тестування просторового алгоритму LSB без завадостійкого кодування

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	0.9524	4.76	122.06	125.09	secretText?
Шум (AWGN) 30 dB	0.0012	99.88	30.29	33.31	?/?z???6????>...
Шум (AWGN) 15 dB	0.0039	99.61	15.75	18.77	?l?e????I?D??...
Фільтр 4000 Hz	0.0215	97.85	85.31	88.34	?P-?#???~??????...
Обрізання 10%	0.9524	4.76	10.0	13.03	secretText?
MP3 Стиснення 128kbps	0.0012	99.88	26.03	29.05	4??E????>?;e??u...
MP3 Стиснення 64kbps	0.001	99.9	26.03	29.06	???W???????f?!...

Таблиця 3.4 — Результати тестування просторового алгоритму LSB із застосуванням коду Хеммінга

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	110.85	113.88	secretText
Шум (AWGN) 30 dB	0.0	100.0	30.24	33.27	2
Шум (AWGN) 15 dB	0.0	100.0	15.71	18.74	[Порожньо]
Фільтр 4000 Hz	0.0	100.0	85.31	88.33	[Порожньо]
Обрізання 10%	1.0	0.0	10.0	13.03	secretText
MP3 Стиснення 128kbps	0.0	100.0	26.03	29.05	? ??däç
MP3 Стиснення 64kbps	0.0	100.0	26.03	29.06	^*4i©

Аналіз результатів тестування просторового методу LSB показує його високу вразливість до деструктивних впливів. Як видно з таблиці 3.3, без використання завадостійкого кодування будь-яке втручання у сигнал призводить до значного пошкодження прихованих даних. Через масову зміну найменш значущих бітів витягнутий водяний знак стає набором випадкових символів, тому в таблиці показано лише його фрагмент, а BER наближається до 100%.

Використання коду Хеммінга, як можна побачити в таблиці 3.4, частково відновлює прихований текст, але лише в простих випадках. Цей код може виправити тільки одну помилку в межах 7-бітного блоку, тому він добре працює при незначних спотвореннях. Однак при MP3-компресії відбувається зміна багатьох бітів одночасно, через що кількість помилок перевищує можливості кодування. У результаті код Хеммінга не справляється з відновленням даних, і інформація втрачається.

3.2 Дослідження стійкості алгоритмів на різних типах аудіосигналів

Щоб перевірити, як різні звуки впливають на якість приховування даних, було обрано три типи аудіофайлів: поліфонічний, низькочастотний та імпульсний сигнали. Як секретне повідомлення використано текст «secretText». Обидва алгоритми тестувалися з використанням методів виправлення помилок, щоб оцінити їхню стійкість до атак.

Вибір саме цих сигналів зумовлений необхідністю перевірити стійкість алгоритмів як у звичайних, так і в екстремальних умовах. Поліфонічний сигнал — це звичайний музичний звук, який містить багато частот одночасно. Цей сигнал дозволяє оцінити роботу алгоритмів у типових для більшості пісень умовах. Низькочастотний сигнал перевіряє роботу алгоритмів у вузькому діапазоні басу. Імпульсний сигнал моделює складніші умови, ніж попередні: він містить ділянки повної тиші, де навіть невеликі зміни можуть викликати помітний шум.

Перший етап тестування проводився на поліфонічному сигналі. Цей аудіоконтейнер характеризується високою щільністю спектра: він містить одночасно низькі, середні та високі частоти, які плавно стихають.

Таблиця 3.5 — Результати тестування просторового алгоритму *LSB (Rich Pad)*

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	104.24	115.29	secretText
Шум (AWGN) 30 dB	0.0	100.0	30.0	41.06	?>?
Шум (AWGN) 15 dB	0.0	100.0	15.0	26.06	ç*?¹
Фільтр 4000 Hz	0.0	100.0	22.79	33.85	DÚÁ×w
Обрізання 10%	1.0	0.0	13.57	24.63	secretText
MP3 Стиснення 128kbps	0.0	100.0	24.66	35.71	?î
MP3 Стиснення 64kbps	0.0	100.0	21.7	32.76	[Порожньо]

Таблиця 3.6 — Результати тестування частотного алгоритму *DWT-SVD (Rich Pad)*

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	28.65	39.71	secretText
Шум (AWGN) 30 dB	1.0	0.0	26.28	37.34	secretText
Шум (AWGN) 15 dB	1.0	0.0	14.82	25.87	secretText
Фільтр 4000 Hz	0.0	100.0	22.45	33.5	
Обрізання 10%	1.0	0.0	13.45	24.51	secretText
MP3 Стиснення 128kbps	1.0	0.0	23.25	34.31	secretText
MP3 Стиснення 64kbps	1.0	0.0	20.81	31.87	secretText

Як можна побачити з таблиці 3.5, що попри високі початкові показники акустичної прозорості, тестування методу на цьому сигналі призвело до сильного пошкодження прихованих даних. Зокрема, при конвертації у MP3 секретне повідомлення втрачається повністю, оскільки кодек прибирає найменш значущі біти для зменшення розміру файлу. Це викликає масові помилки, які код Хеммінга вже не може виправити.

Натомість дані з таблиці 3.6 показують, що алгоритм DWT-SVD є стійким до шуму та MP3-компресії, але вразливим до низькочастотної фільтрації. Згідно з архітектурою методу, дані вбудовуються в низькочастотну частину сигналу, що забезпечує їх часткову стійкість до стиснення. Проте поліфонічний сигнал має широкий спектр частот. Під час застосування фільтра Баттерворта з порогом зрізу 4000 Гц видаляється значна частина високочастотних компонентів, які також впливають на загальну структуру сигналу. Оскільки сингулярні числа в методі SVD залежать від розподілу енергії в матриці сигналу, така фільтрація змінює їхні значення та призводить до часткового руйнування прихованого водяного знака.

Другий етап експертизи базується на використанні низькочастотного сигналу частотою 50 Гц.

Таблиця 3.7 — Результати тестування просторового алгоритму LSB (Sub-Bass)

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	101.92	117.15	secretText
Шум (AWGN) 30 dB	0.0	100.0	30.01	45.24	÷İp¿6OU???ÙÁ
Шум (AWGN) 15 dB	0.1429	85.71	14.99	30.23	©ÜrÓ
Фільтр 4000 Hz	0.0	100.0	50.33	65.56	íA
Обрізання 10%	1.0	0.0	13.77	29.0	secretText
MP3 Стиснення 128kbps	1.0	0.0	26.02	41.25	secretText
MP3 Стиснення 64kbps	1.0	0.0	25.95	41.18	secretText

Таблиця 3.8 — Результати тестування частотного алгоритму DWT-SVD (Sub-Bass)

Експеримент	NCC	BER (%)	SNR (dB)	PSNR (dB)	Витягнутий текст
Еталон (Без атак)	1.0	0.0	24.32	39.55	secretText
Шум (AWGN) 30 dB	1.0	0.0	23.27	38.5	secretText
Шум (AWGN) 15 dB	1.0	0.0	14.5	29.74	secretText
Фільтр 4000 Hz	1.0	0.0	29.26	44.49	secretText
Обрізання 10%	1.0	0.0	13.43	28.67	secretText
MP3 Стиснення 128kbps	1.0	0.0	22.34	37.57	secretText
MP3 Стиснення 64kbps	1.0	0.0	21.6	36.83	secretText

Обрізання 10%	1.0	0.0	10.11	13.18	secretText
MP3 Стиснення 128kbps	1.0	0.0	21.71	24.79	secretText
MP3 Стиснення 64kbps	1.0	0.0	21.16	24.24	secretText

Аналіз даних таблиці 3.9 показує, що у даному тестуванні метод LSB не зміг відновити секретне повідомлення в таких умовах. Оскільки метод змінює наймолодші біти, у нульових значеннях з'являються одиниці. Це викликає появу високочастотного шуму на ділянках, які мали б залишатися беззвучними. Попри високий SNR, MP3-кодек сприймає такі зміни як спотворення і частково їх видаляє. У результаті водяний знак сильно пошкоджується, а BER досягає 90–100%, що унеможлиблює відновлення тексту.

А результати таблиці 3.10 показують високу стійкість алгоритму DWT-SVD до таких умов. Оскільки вейвлет-перетворення враховує розподіл енергії сигналу, дані вбудовуються в ділянки сигналу з більшою амплітудою, а не в тишу. Це зменшує вплив порожніх фрагментів. Завдяки використанню коду Ріда-Соломона система додатково підвищує стійкість і забезпечує повне відновлення даних без помилок.

3.3 Підсумковий аналіз результатів тестування

Після проведення тестування на поліфонічному, низькочастотному та імпульсному сигналах було зроблено такі висновки.

Алгоритм LSB продемонстрував низьку стійкість до зашумлення та транскодування. Основною причиною цього є його залежність від найменш значущих бітів, які можуть бути змінені або втрачені під час роботи психоакустичних моделей стиснення. Перевагою LSB є його висока швидкодія, низька обчислювальна складність та висока акустична прозорість за умови відсутності атак. У випадку низькочастотного сигналу алгоритм працює краще через надлишковість спектрального представлення, але такі умови не є типовими для реального аудіо.

Натомість алгоритм DWT-SVD показав значно вищу стійкість у більшості випадках. Використання вейвлет-перетворення дозволяє враховувати структуру сигналу, що покращує стійкість до шуму та MP3-компресії. Алгоритм вбудовує інформацію в більш стабільні частини спектра, завдяки чому зменшується вплив більшості типових атак. Проте, на відміну від LSB, цей метод потребує значних затрат процесорного часу на складні матричні обчислення, а також має меншу ємність для приховування даних. Водночас продемонстровано, що метод залишається чутливим до низькочастотної фільтрації у випадках, коли значна частина спектральної інформації видаляється.

Проведені експерименти також підтвердили важливість використання алгоритмів завадостійкого кодування. Код Хеммінга показав низьку ефективність у випадках масових спотворень, оскільки він розрахований на виправлення поодиноких помилок. Натомість код Ріда-Соломона продемонстрував високу ефективність у випадках із пакетними помилками, забезпечуючи коректне відновлення даних.

ВИСНОВКИ

Підсумовуючи, використання просторового алгоритму LSB є доцільним лише для передачі даних в ідеальних, безвтратних каналах зв'язку. Гібридний метод DWT-SVD довів свою ефективність як надійний інструмент для захисту авторського права на мультимедійні файли, що робить його кращим для практичного застосування. Класичні просторові алгоритми є вразливими до стиснення та шуму, тому погано підходять для сучасної передачі медіаданих. Натомість частотні методи краще враховують структуру сигналу і мають вищу стійкість до більшості завад.

Водночас повністю гарантувати цілісність прихованої інформації лише за допомогою стеганографії неможливо. Надійність систем досягається лише при використанні комбінованого підходу, де частотне вбудовування поєднується з алгоритмами корекції помилок. Така схема дозволяє відновлювати дані після стиснення та шумових впливів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Salson M. Forward Error Correction and Burst Errors [Electronic resource] / M. Salson // HAL Theses. – Available from: <https://theses.hal.science/tel-04918178/document>
2. DeepSound – Audio Steganography [Electronic resource]. – Available from: <https://github.com/Jpinsoft/DeepSound>
3. SilentEye – Cross-platform steganography tool [Electronic resource]. – Available from: <https://achorein.github.io/silenteye/>
4. Streamlit Official Documentation [Electronic resource]. – Available from: <https://docs.streamlit.io/>
5. REST API: What it is and how it works [Electronic resource] // IBM. – Available from: <https://www.ibm.com/topics/rest-apis>
6. NumPy Documentation [Electronic resource]. – Available from: <https://numpy.org/doc/>
7. SciPy Reference Guide [Electronic resource]. – Available from: <https://docs.scipy.org/doc/scipy/>
8. PyWavelets [Electronic resource]. – Available from: <https://pywavelets.readthedocs.io/>
9. Reedsolo Codec [Electronic resource] // PyPI. – Available from: <https://pypi.org/project/reedsolo/>
10. Django Documentation [Electronic resource] // Django Project. – Available from: <https://docs.djangoproject.com/>
11. Scipy.signal.butter [Electronic resource] // SciPy Reference Guide. – Available from: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.butter.html>

12. Audio Steganography: LSB & DWT-SVD: source code of the program [Electronic resource]. – Available from: <https://github.com/VeroniikaPeleshchak/audio-steganography>
13. Katzenbeisser S. Information Hiding Techniques for Steganography and Digital Watermarking / S. Katzenbeisser, F. Petitcolas. – Artech House Inc., 2000. – 210 p.
14. Spanias A. Audio Signal Processing and Coding / A. Spanias, T. Painter, V. Atti. – Wiley-Interscience, New Jersey, 2006. – 541 p.
15. Stein J. Y. Digital Signal Processing: a Computer Science Perspective / J. Y. Stein. – Wiley-Interscience, New York, 2000. – 856 p.
16. Bender W. Techniques for Data Hiding / W. Bender, D. Gruhl, N. Morimoto, A. Lu // IBM Systems Journal. – 1996. – Vol. 35. – P. 187-203.
17. Cox I. J. Digital Watermarking and Steganography / I. J. Cox, M. L. Miller, J. A. Bloom, J. Fridrich, T. Kalker. – 2nd ed. – Morgan Kaufmann, 2007. – 619 p.